

Contents

TravelView 真机测试环境搭建 (iMac M1 + iPhone 12 Pro Max + OPPO Android)	1
0. 先说结论：整体拓扑	1
1. 照片安全：四道防线	2
防线 1 权限层：只申请「读」，连写的能力都不存在	2
防线 2 代码层：把写 API 从代码库里禁掉	2
防线 3 数据层：EXIF 不落盘、原图不搬运	2
防线 4 物理兜底	3
2. iMac M1 环境搭建（一次性）	3
3. 连接 iPhone 12 Pro Max	3
4. 连接 OPPO Android	4
5. 日常开发循环	4
6. 产物自动回到 iMac 的 test-output/	4
6.1 启动接收服务（在 iMac 上）	4
6.2 让手机能连上	5
6.3 App 侧（dev 专用，release 编译期剔除）	5
6.4 备用方案（不想开服务时）	5
7. 项目目录约定	5
8. 建议的推进顺序	6

TravelView 真机测试环境搭建 (iMac M1 + iPhone 12 Pro Max + OPPO An-

三个目标：①一条命令编译安装到两台真机②你的照片物理上不可能被修改或外传③手机上跑出来的产物自动落到 iMac 的 test-output/

0. 先说结论：整体拓扑

```

graph TD
    subgraph Mac_M1 [iMac M1 (开发机 + 产物接收方)]
        direction TB
        M1_1[Flutter SDK / Xcode / Android SDK]
        M1_2[tools/devsink ← 本地 HTTP 服务 :8787]
        M1_3[test-output/runs/<时间戳> ← 产物落地]
    end

    subgraph Mobile_Devices [USB / Wi-Fi]
        direction TB
        subgraph iPhone_12_PM [iPhone 12 PM]
            direction TB
            I12P_1[只读相册]
            I12P_2[产物 POST 回]
        end
        subgraph OPPO_Android [OPPO Android]
            direction TB
            OPA_1[只读相册]
            OPA_2[产物 POST 回]
        end
    end

    Mac_M1 --- Mobile_Devices
  
```

关键点：手机 App 无法直接写 iMac 的文件夹。所以 dev 版本内置一个”产物回传”开关，把生成的 JSON/图片/网页 POST 到你 Mac 上的本地服务，由它写进 test-output/。这条通道只在 **debug** 构建里存在，release 构建里编译期就被剔除。

1. 照片安全：四道防线

你的顾虑完全合理——这是唯一不能出错的地方。按下面四层做，即使代码写错了也伤不到照片。

防线 1 | 权限层：只申请「读」，连写的能力都不存在

iOS — `ios/Runner/Info.plist` 只放这一条：

```
<key>NSPhotoLibraryUsageDescription</key>
<string> 用于读取旅行照片的时间与位置，生成行程路线。App 不会修改或删除任何照片。</string>
```

绝对不要加 `NSPhotoLibraryAddUsageDescription`（写入相册权限）。Flutter 侧请求权限时显式声明只读：

```
final ps = await PhotoManager.requestPermissionExtend(
  requestOptions: const PermissionRequestOption(
    iosAccessLevel: IosAccessLevel.read, // ← 只读, 不是 readWrite
  ),
);
```

系统层面 App 就没有写相册的授权，`PHPhotoLibrary` 的任何写操作会直接失败。

Android — `AndroidManifest.xml` 只放这两条：

```
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>
<uses-permission android:name="android.permission.ACCESS_MEDIA_LOCATION"/>
```

绝不能出现：`WRITE_EXTERNAL_STORAGE`、`MANAGE_EXTERNAL_STORAGE`、`READ_MEDIA_VIDEO` (V1 用不到)。Android 10+ 的分区存储下，App 修改别人创建的媒体文件必须经 `MediaStore.createWriteRequest()` 弹窗确认——只要代码里没有这个调用，就没有任何静默修改路径。

防线 2 | 代码层：把写 API 从代码库里禁掉

`photo_manager` 有 `PhotoManager.editor.*`（删除、保存到相册）。全项目只允许一个文件访问相册：

`lib/data/photo/photo_repository.dart` ← 唯一接触 `photo_manager` 的文件

里面只用 `getAssetPathList / getAssetListPaged / thumbnailDataWithSize / file`（只读）。

加一个 pre-commit 钩子，出现禁用符号直接拒绝提交：

```
# .githooks/pre-commit
BANNED=
↪ 'PhotoManager.editor|deleteWithIds|saveImage|saveVideo|MANAGE_EXTERNAL_STORAGE|WRITE_
if git diff --cached -U0 | grep -nE "$BANNED"; then
  echo "[禁止] 检测到相册写入相关代码/权限，已阻止提交"; exit 1
fi
```

启用：`git config core.hooksPath .githooks && chmod +x .githooks/pre-commit`

防线 3 | 数据层：EXIF 不落盘、原图不搬运

- 扫描阶段只读元数据（id / 时间 / 经纬度），不解码图片、不复制文件。
- 需要图片时走 `AssetEntity.file`（系统返回沙盒内的临时副本，原图纹丝不动）。
- 回传到 Mac 的图片一律是压缩后的 **WebP** 且已剥离 **EXIF**。

防线 4 | 物理兜底

正式开测之前，两台手机各做一次完整备份（iPhone → iCloud 照片或 Finder 整机备份；OPPO → Google 相册或导出到 iMac 一份）。这是 15 分钟的事，但让你后面所有实验都可以放心大胆。

顺带一个提速技巧：第一次真机扫描后，把脱敏的元数据导出成 JSON 存到 `test-output/fixtures/`（只有时间戳 + 经纬度，没有任何图片）。之后开发聚类算法、路线渲染、页面模板，全部在 iMac 上跑纯 Dart 单元测试，根本不用连手机。真机只在验证相册读取和最终效果时才需要——照片被碰到的次数从“每次改代码”降到“一周两三次”。

2. iMac M1 环境搭建（一次性）

```
# 基础
xcode-select --install
brew install --cask android-studio
brew install cocoapods fvm jq

# Flutter (用 fvm 管版本, 避免以后和 TensuGo 的 Flutter 版本打架)
fvm install stable && fvm use stable --force

# JDK 17 (Android Gradle 需要)
brew install --cask temurin@17
```

Xcode: App Store 装最新版 → 打开一次同意协议 → `sudo xcodebuild -license accept` **Android SDK:** Android Studio → SDK Manager 里勾选 Platform-Tools、Build-Tools、最新 Platform，并把 `~/Library/Android/sdk/platform-tools` 加进 PATH。

最后：

```
fvm flutter doctor -v      # 直到全绿 (除 Chrome/Web 可忽略)
```

M1 上的两个常见坑：CocoaPods 要装 arm64 版（用 brew 装的就是对的，别用系统 Ruby 的 gem）；Xcode 里若报 arm64 simulator 链接错误，在 Podfile 的 `post_install` 里排除 `EXCLUDED_ARCHS[sdk=iphonesimulator*] = i386`。

3. 连接 iPhone 12 Pro Max

1. **Apple 开发者账号：**免费账号即可，但签名 7 天过期，每周要重新 flutter run 一次。如果你打算持续开发几个月，\$99/年的账号（证书 1 年有效 + 可用 TestFlight）会省掉大量重签的烦躁。建议先免费跑通，确认项目做得下去再付费。
2. **手机端：**数据线连 Mac → 手机上点「信任此电脑」→ 设置 → 隐私与安全性 → 打开 开发者模式（需重启）。
3. **Xcode 签名：**用 Xcode 打开 `app/ios/Runner.xcworkspace` → Signing & Capabilities → 勾 Automatically manage signing → Team 选你的 Apple ID → Bundle ID 改成唯一的（如 `com.tuxy.travelview.dev`）。
4. **首次安装后：**手机设置 → 通用 → VPN 与设备管理 → 信任你的开发者证书。
5. 验证：

```
fvm flutter devices          # 应该看到 iPhone
fvm flutter run -d <iphone-id>
```

6. 无线调试: Xcode → Window → Devices and Simulators → 勾选“Connect via network”, 之后拔掉线也能 run。

4. 连接 OPPO Android

1. 设置→关于手机→连点「版本号」7次开启开发者选项。
2. 开发者选项里打开: **USB 调试**、**USB 安装** (允许通过 **USB 安装应用**)。
3. 注意 - **ColorOS** 特有的坑:「USB 安装」这个开关通常要求先登录 **OPPO/HeyTap** 账号才能打开, 而且有时需要插着数据线才出现该选项。这一步卡住的人很多, 提前有心理准备。
4. 另外关掉开发者选项里的「权限监控 / 禁止权限监控」, 否则 debug 包容易被后台清理。
5. 验证:

```
adb devices          # 手机上会弹「允许 USB 调试」, 勾选「一律允许」
fvm flutter run -d <android-id>
```

6. 无线调试 (Android 11+, 推荐, 省得一直插线):

```
adb pair <手机显示的 IP: 配对端口>    # 开发者选项 → 无线调试 → 使用配对码配对设备
adb connect <手机 IP: 调试端口>
```

5. 日常开发循环

```
cd app

fvm flutter run -d <id>          # 装上并进入热重载; 改代码按 r 热重载, R 热重启
fvm flutter run -d all          # 两台手机同时装、同时热重载 (对比 iOS/Android 差
↪ 异神器)
fvm flutter run --release -d <id> # 测真实性能 (扫几千张照片时必须用 release 测)
fvm flutter run --dart-define=DEVSINK=http://192.168.x.x:8787 # 指定产物回收地址
```

热重载覆盖 90% 的 UI 改动, 秒级生效, 不需要重新安装。只有改了原生代码/权限/依赖才要重新 run。

6. 产物自动回到 iMac 的 test-output/

6.1 启动接收服务 (在 iMac 上)

```
cd tools/devsink
python3 server.py          # 监听 :8787, 写入 ../../test-output/runs/< 时间戳 >/
```

(tools/devsink/server.py 我已经写好放在项目里了, 零依赖, 直接跑。)

6.2 让手机能连上

- **Android 走 USB** (最稳, 不依赖 Wi-Fi):

```
adb reverse tcp:8787 tcp:8787      # 手机上的 localhost:8787 == Mac 的 8787
```

App 里 DEVSINK 填 `http://127.0.0.1:8787`。

- **iPhone 走 Wi-Fi** (iOS 没有 adb reverse): 手机和 Mac 连同一个 Wi-Fi, 取 Mac 的局域网 IP:

```
ipconfig getifaddr en0
```

DEVSINK 填 `http://< 那个 IP>:8787`。注意: iOS 默认禁止明文 HTTP, 只在 **debug** 的 **Info.plist** 里加 `NSAllowsLocalNetworking` (不是 `NSAllowsArbitraryLoads`), 并且首次会弹「查找并连接本地网络设备」权限, 要允许。

6.3 App 侧 (dev 专用, release 编译期剔除)

```
const _sink = String.fromEnvironment('DEVSINK');

Future<void> exportToMac(String runName, Map<String, List<int>> files) async {
  if (!kDebugMode || _sink.isEmpty) return; // release 包里这段被 tree-shake 掉
  for (final e in files.entries) {
    await dio.post('$_sink/upload',
      data: Stream.fromIterable([e.value]),
      options: Options(headers: {
        'X-Run': runName,           // 例如 kyoto-2025-09
        'X-Path': e.key,           // 例如 manifest.json / photos/001.webp
        'Content-Length': e.value.length,
      }));
  }
}
```

6.4 备用方案 (不想开服务时)

- Android: `adb pull /sdcard/Android/data/< 包名 >/files/out ./test-output/runs/`
- iOS: Xcode → Devices and Simulators → 选 App → 齿轮 → Download Container (需在 Info.plist 开 `UIFileSharingEnabled`)

7. 项目目录约定

TravelView/	
└── app/	Flutter 工程
└── server/	Go 后端 (后续)
└── web/	分享站 (后续)
└── tools/devsink/	产物接收服务 ← 已就绪
└── test-output/	(!) 已在 .gitignore, 绝不进版本库
└── fixtures/	脱敏元数据 JSON (可安全提交, 供单测用)
└── runs/2026-09-04-1530/	每次真机跑出来的产物

└── Design/ architecture.md / dev-setup.md

.gitignore 里务必加:

```
test-output/runs/  
*.heic  
*.jpg  
*.png  
!Design/**/*.*png
```

防止某次手滑把真实旅行照片提交上去——这比相册被改更容易发生。

8. 建议的推进顺序

步	做什么	完成标志
1	装环境, flutter doctor 全绿	—
2	空 Flutter 工程分别跑上两台真机	两台手机都出现 hello world
3	起 devsink, App 里点个按钮回传一个 hello.txt	test-output/runs/ 里出现文件
4	加只读相册权限 + 四道防线, 读出某个日期范围的照片 数量	数字对得上
5	读出经纬度 + 时间, 导出 JSON 到 Mac	fixtures 里有真实数据
6	之后的算法开发全部在 Mac 上对着 fixtures 写单测	不再频繁碰手机

第 4 步在 OPPO 上大概率会返回全空的经纬度 (ACCESS_MEDIA_LOCATION + setRequireOriginal 的坑), 这是预期内的, 也是整个项目最该先解决的技术问题。